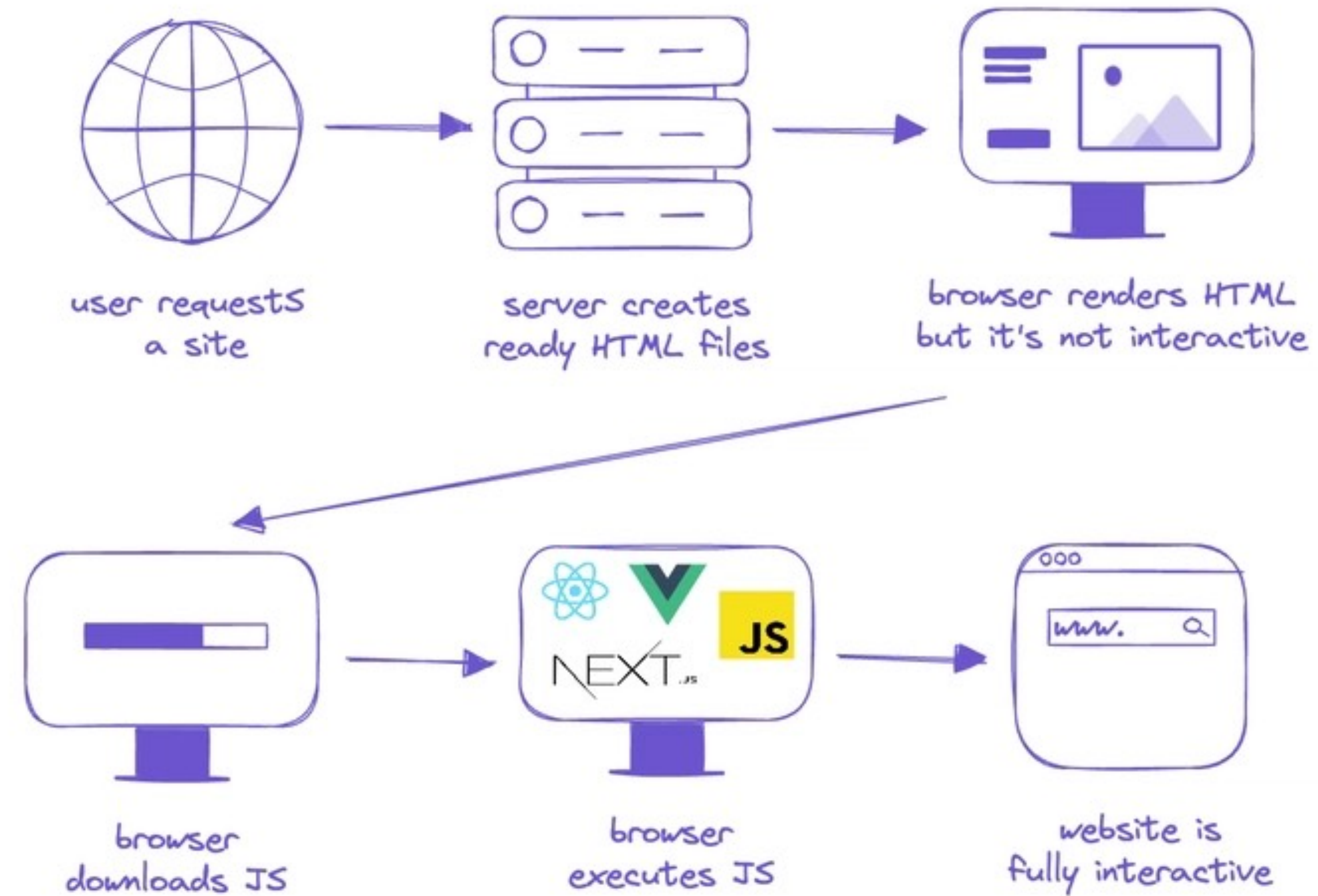


SERVER FUNCTIONS

I. Lesson

Server-side Rendering (SSR)



Workspace members



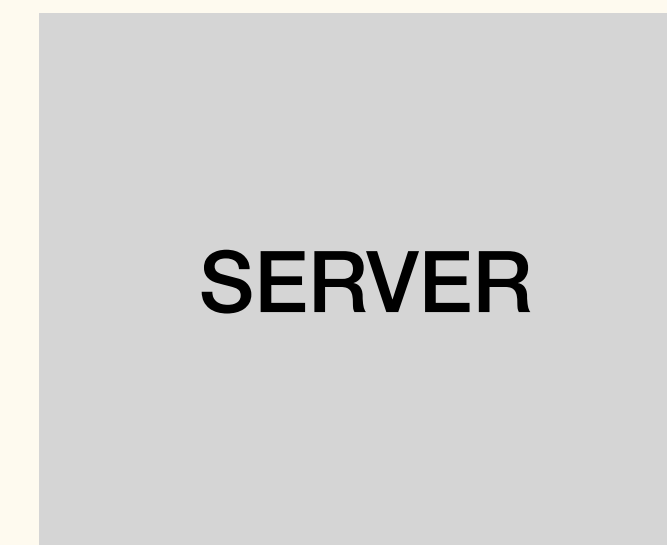
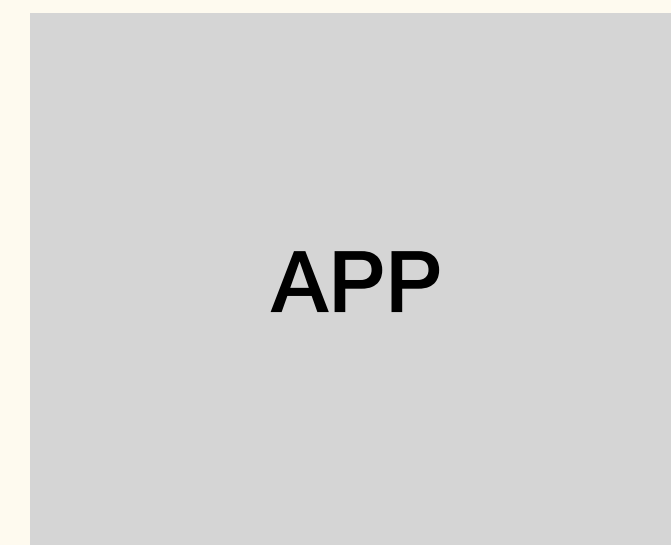
```
[workspace]
members = ["app", "server"]
resolver = "2"

// ...

[workspace.dependencies]
// ...

[[workspace.metadata.leptos]]
// .....
```

WORKSPACE



II. Project

STRUCTURE OF THE PROJECT

```
● ● ●  
.app/  
├─ Cargo.toml  
├─ src  
│   ├── app.rs  
│   ├── components  
│   │   ├── mod.rs  
│   │   ├── navbar.rs  
│   │   ├── reactive_indicator.rs  
│   │   ├── theme_toggle.rs  
│   │   └─ ui  
│   │       ├── button.rs  
│   │       ├── card.rs  
│   │       └─ mod.rs  
│   ├── lib.rs  
│   ├── routing  
│   │   ├── mod.rs  
│   │   ├── page_home.rs  
│   │   └─ page_test.rs  
│   └─ shell.rs
```

```
● ● ●  
.server/  
├─ Cargo.toml  
├─ src  
│   ├── fileserv.rs  
│   └─ main.rs
```

1. APP

1. App —> Cargo.toml

```
[package]
name = "app"
version = "0.1.0"
edition = "2024"

[lib]
crate-type = ["cdylib", "rlib"]

[dependencies]
// ....

[features]
default = []
hydrate = ["leptos/hydrate"]
ssr = [
    "leptos/ssr",
    "leptos_meta/ssr",
    "leptos_router/ssr",
    "dep:leptos_axum",
]
```

1. App —> shell.rs

```
● ● ●

use leptos::prelude::*;

use crate::app::App;

pub fn shell(leptos_options: LeptosOptions) -> impl IntoView {
    view! {
        <!DOCTYPE html>
        <html lang="en">
            <head>
                <meta charset="utf-8" />
                <title text="Title" />
                <link rel="shortcut icon" type_="image/png" href="/favicon.png" />
                <link id="leptos" href="/pkg/app_name.css" rel="stylesheet" />
                <AutoReload options=leptos_options.clone() />
                <HydrationScripts options=leptos_options />
            </head>

            <body>
                <App />
            </body>
        </html>
    }
}
```

1. App —> lib.rs



```
#![warn(clippy::all)]
#![deny(clippy::unwrap_used)]

pub mod routing;

pub mod app;
pub mod components;
pub mod shell;

#[cfg(feature = "hydrate")]
#[wasm_bindgen::prelude::wasm_bindgen]
pub fn hydrate() {
    use app::App;
    _ = console_log::init_with_level(log::Level::Debug);
    console_error_panic_hook::set_once();

    leptos::mount::hydrate_body(App);
}
```


2. Server

2. Server —> Cargo.toml



```
[package]
name = "server"
version = "0.1.0"
edition = "2024"

[dependencies]
app = { path = "../app", default-features = false, features = ["ssr"] }
leptos = { workspace = true, features = [ "ssr" ]}
leptos_axum.workspace = true

axum.workspace = true
tokio.workspace = true
tower.workspace = true
tower-http.workspace = true
log.workspace = true
```

2. Server —> main.rs

```
#[tokio::main]
async fn main() {
    let conf = get_configuration(None).unwrap();
    let addr = conf.leptos_options.site_addr;
    let leptos_options = conf.leptos_options;
    // Generate the list of routes in your Leptos App
    let routes = generate_route_list(App);

    let app = Router::new()
        .leptos_routes(&leptos_options, routes, {
            let leptos_options = leptos_options.clone();
            move || shell(leptos_options.clone())
        })
        .fallback(leptos_axum::file_and_error_handler(shell))
        .with_state(leptos_options);

    log!("listening on http://{addr}", &addr);
    let listener = tokio::net::TcpListener::bind(&addr).await.unwrap();
    axum::serve(listener, app.into_make_service())
        .await
        .unwrap();
}
```


COMMON PATTERNS

Server functions



```
#[server]
pub async fn greet(name: String) -> Result<String, ServerFnError> {
    // This code only runs on the server!
    Ok(format!("Hello, {name}! Greetings from the server!"))
}
```

Action::new()



```
let name_signal = RwSignal::new(String::new());
let greeting_action = Action::new(|name: &String| {
    let name = name.clone();
    async move { greet(name).await }
});

let on_greet = move |_| {
    greeting_action.dispatch(name_signal.get());
};
```

Let's code!

Your turn!